

Two dimensionless ratios define operational limits in hierarchical-memory inference

Geunsik Lim ^{1*}

¹ College of Information and Communication Engineering,
Sungkyunkwan University (SKKU), Suwon, South Korea .

Corresponding author(s). E-mail(s): leemgs@g.skku.edu;

Abstract

Hierarchical-memory inference conflates two constraints: whether an active working set fits in fast memory and whether computation can hide compulsory cross-tier transfer. We separate them with dimensionless residency $R_C = C_{\text{fast}}/W$ and overlap $R_B = T_{\text{comp}}/T_{\text{transfer}}$. The pair yields normalized lower bounds on non-resident data and exposed transfer, and an auditable measurement protocol rather than a device-specific performance predictor. A dependent-load probe on one Intel Xeon CPU shows working-set sensitivity without a discontinuity at the declared $R_C = 0.5$ convention. Layered-matrix probes on three accelerators of two vendors (NVIDIA Tesla T4, NVIDIA A100, and Google TPU v5e) show higher latency under low residency ($1.23\text{--}2.83\times$); the Tesla T4 sweep also crosses the derived $R_B = 1$ overlap boundary. The contribution is a falsifiable, scale-independent reporting discipline linking capacity and overlap evidence. Production workloads, per-device boundaries, and population-level hardware generality remain open.

Keywords: hierarchical memory systems, memory orchestration, AI inference, working-set residency, compute-transfer overlap, reproducible measurement

1 Introduction

Modern foundation models have shifted the performance frontier of AI inference from arithmetic throughput to *data coordination*. When model weights, activations, and key-value (KV) caches exceed accelerator memory, inference latency is governed less by compute and more by how efficiently execution coordinates data movement across a hierarchical memory system spanning device memory, host memory, and

secondary storage [1, 2]. Because inference now dominates the recurring cost and energy of deployed models, this coordination has direct economic and environmental stakes [3–5]. Memory-bound inference spans autoregressive language models [6, 7], vision and vision–language models [8, 9], and retrieval-augmented pipelines [10], so the coordination question recurs across otherwise different workloads.

Hardware analyses identify memory capacity, bandwidth, and interconnect latency as important constraints on inference [11–13]. Separating capacity pressure from exposed transfer is therefore useful when reporting and comparing inference systems.

Existing systems reduce memory pressure along several axes. Serving frameworks manage the KV cache and schedule requests to raise throughput [14–17]; offloading and memory-pooling systems spill weights or activations to host DRAM or NVMe [18–23]; swapping systems stage tensors on and off the accelerator [24–26]; and compression—sparsity, pruning, and IO-aware attention—shrinks the working set or its traffic [27–29]. Parallelism redistributes memory across devices [30, 31], while host-side allocators and NUMA-aware placement shape the sustained transfer path [32–35]. The design space is large enough to warrant several recent surveys [36, 37], and it widens further in edge and heterogeneous deployments [38–41]. Across these approaches the same technique can help or hurt depending on model size, batch size, hardware topology, and how much compulsory traffic remains on the critical path. This motivates a compact description that exposes those conditions before comparing policies.

We take a regime-based view. Denning’s working-set model and thrashing theory [42] identify capacity saturation as a binary event on a *single-tier* system, and the Roofline model [43] expresses performance as $\min(F \cdot I, B_{\text{peak}})$ for a fixed workload on a single tier. Roofline supplies a throughput ceiling and working-set analysis supplies a capacity lens, but neither alone distinguishes two operating points with the same arithmetic intensity and working-set size when one can overlap its cross-tier traffic and the other cannot. Conversely, a queueing description can represent overlap but does not supply a portable coordinate for residency. This is the narrow gap addressed here: not a replacement for these frameworks, but a common coordinate system for a *multi-tier* hierarchy in which locality, cross-tier transfer, and synchronisation co-exist. Building instead on classical queueing analysis of overlapping services [44], our accounting model decomposes end-to-end latency into four terms ($T_{\text{comp}} + T_{\text{mem}} + T_{\text{swap}} + T_{\text{sync}}$) and uses two dimensionless ratios to label an operating point. The three operational labels are governed by the fast-memory residency ratio $R_C = C_{\text{fast}}/W$ and the overlap ratio $R_B = B_{\text{slow}}T_{\text{comp}}/D = T_{\text{comp}}/T_{\text{transfer}}$. The labels state whether majority residency is available and whether ideal compute–transfer overlap can hide compulsory traffic; they do not by themselves predict a policy’s speed-up.

The advance is deliberately analytical and methodological, with empirical evidence used to test direction rather than to claim a production effect size. We make three contributions:

- **An identifiable operating coordinate.** We separate capacity pressure from exposed transfer using independently measurable R_C and R_B , avoiding the circular use of total latency to explain itself. The continuous coordinates remain reportable when a categorical threshold is inappropriate.

- **Bounds that map diagnosis to intervention.** We derive a conservative service-time bound, the minimum non-resident share $\max(0, 1 - R_C)$, and the transfer share that ideal overlap cannot hide $\max(0, 1 - R_B)$. Only $R_B = 1$ is a derived crossover; $R_C = 0.5$ is a transparent majority-residency convention whose sensitivity must be reported rather than fitted away.
- **A falsifiable open protocol and bounded evidence.** A compiled CPU probe and layered-matrix runs on 3 accelerators of two vendors test the predicted directions. Residency sensitivity appears on all three accelerators and the overlap crossover is resolved on the Tesla T4; code and records make stronger future tests directly comparable.

Together, the formulation and measurements provide a reproducible starting point for testing when hierarchical-memory coordination is constrained by residency or exposed transfer. They confirm the residency direction on one CPU and three accelerators, and the overlap boundary on one GPU, but do not establish production effect sizes, strategy rankings, or per-device boundary values. The scientific value is the separation of two necessary constraints that can otherwise generate the same observed latency, plus a protocol for rejecting the formulation when those directional predictions fail.

2 Results

2.1 A two-ratio operational model

We describe a hierarchical-memory operating point using

$$R_C = \frac{C_{\text{fast}}}{W}, \quad R_B = \frac{B_{\text{slow}} T_{\text{comp}}}{D} = \frac{T_{\text{comp}}}{T_{\text{transfer}}}, \quad (1)$$

where C_{fast} is usable fast-memory capacity, W is the active working set, D is compulsory cross-tier traffic per step, B_{slow} is sustained bandwidth on the limiting link, and $T_{\text{transfer}} = D/B_{\text{slow}}$. Unlike a ratio that uses total step time, R_B is not circular: it compares independently timed computation and transfer intervals.

We use $\theta_C = 0.50$ as an explicit majority-residency convention and derive $\theta_B = 1$ from equality of computation and transfer time. These definitions produce three operational labels:

- **capacity-limited:** $R_C < \theta_C$;
- **I/O-limited:** $R_C \geq \theta_C$ and $R_B < \theta_B$; and
- **coordination-dominated:** $R_C \geq \theta_C$ and $R_B \geq \theta_B$.

The capacity boundary is a convention, not a thermodynamic critical point. The I/O boundary is an overlap boundary: for $R_B < 1$, at least $T_{\text{transfer}} - T_{\text{comp}}$ cannot be hidden by ideal overlap. Primary analyses should therefore retain continuous (R_C, R_B) values. A label near $R_C = 0.5$ is convention-sensitive, whereas crossing $R_B = 1$ changes the sign of the ideal-overlap remainder. This distinction prevents a convenient residency cut-off from being presented as an empirically discovered constant.

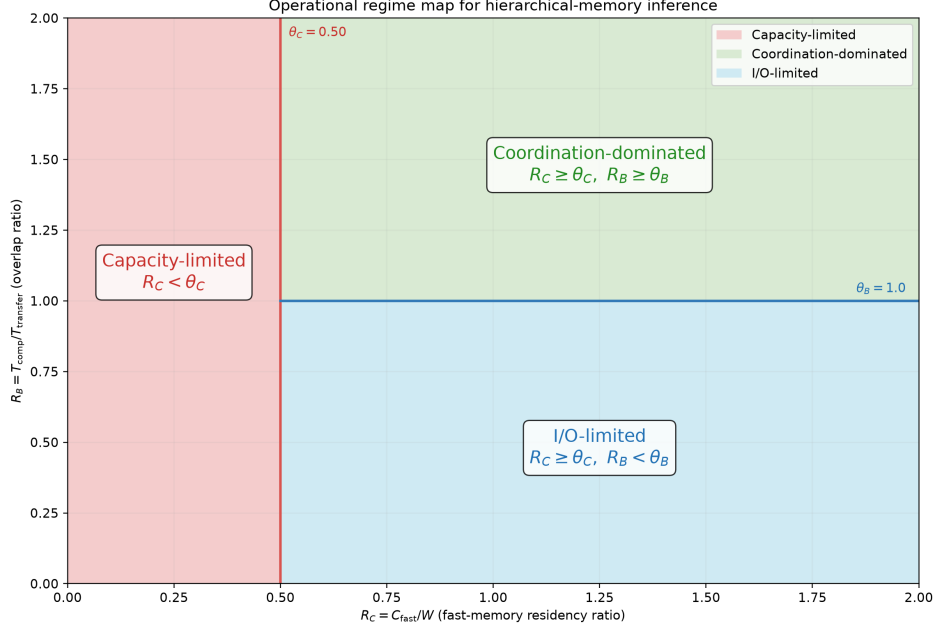


Fig. 1 Analytical regime map. The vertical line is the declared majority-residency convention $\theta_C = 0.50$; the horizontal segment is the derived overlap boundary $\theta_B = 1.0$. The diagram is generated from the same constants as the released classifier and contains no empirical data.

2.2 Irreducible costs and falsifiable predictions

For accounting purposes, define non-overlapping critical-path contributions so that a step can be written as

$$T_{\text{total}} = T_{\text{comp}} + T_{\text{mem}} + T_{\text{swap}} + T_{\text{sync}}. \quad (2)$$

Independently of that accounting decomposition, a conservative service-time lower bound that allows the underlying services to overlap is

$$T_{\text{total}} \geq \max \left\{ T_{\text{comp}}, \rho W \max(0, 1 - R_C), \frac{D}{B_{\text{slow}}} \right\}, \quad (3)$$

where W denotes bytes accessed in the step and ρ is a lower bound on service time per non-resident byte for the specified access pattern. Taking the maximum permits ideal overlap and avoids double-counting bytes that contribute to both a cache miss and a cross-tier transfer. Synchronisation and contention can only increase observed time, so Equation (3) is not a performance predictor. The residual above it cannot be assigned to one latency component without event-level measurements.

Two normalized consequences make the formulation useful beyond relabelling a roofline plot. No placement policy can make more than C_{fast} bytes resident, so the

124 minimum non-resident share of an active working set is

$$f_{\text{nonresident}} \geq \max(0, 1 - R_C). \quad (4)$$

125 Even a perfect overlap schedule leaves an exposed interval $\max(0, T_{\text{transfer}} - T_{\text{comp}})$;
 126 normalized by transfer time,

$$f_{\text{exposed}} \geq \max(0, 1 - R_B). \quad (5)$$

127 Thus R_C answers how much placement pressure is irreducible, whereas R_B answers
 128 how much compulsory-transfer service cannot be hidden. The pair also supports controlled
 129 comparison across scales: multiplying both capacity and working set by the
 130 same factor leaves R_C unchanged, as does multiplying both isolated service times by
 131 the same factor for R_B . Absolute latency still requires the dimensional quantities in
 132 Equation (3).

133 This separation gives an actionable diagnostic without asserting an optimal policy.
 134 If $R_C < 0.5$, interventions must reduce the active working set, increase usable capacity,
 135 or reduce the bytes touched while non-resident. If $R_C \geq 0.5$ but $R_B < 1$, overlap
 136 alone cannot remove the exposed interval, so reducing compulsory traffic or increasing
 137 sustained link bandwidth is necessary. If both tests pass, the residual directs measurement
 138 toward synchronisation, launch, contention, or locality costs. Labels describe
 139 operating points, not devices, and can change after an intervention.

140 Both ratios are computable from datasheets before any run, which makes the diagnostic
 141 usable at planning time. For example, a 70B-parameter model holds roughly
 142 140 GB of weights in `fp16`. Deployed on a single 40 GB accelerator, $R_C = 40/(140 + K) \leq 40/140 \approx 0.29 < 0.5$ for any KV/activation footprint $K \geq 0$: the configuration is capacity-limited, and no offloading schedule can remove the majority-eviction cost—additional fast capacity is required. Spreading the same weights across four such accelerators ($C_{\text{fast}} = 160$ GB) gives $R_C \approx 1$, leaving the capacity-limited region. This placement conclusion follows from published capacities and parameter counts alone,
 148 and is exactly the kind of decision the two ratios are meant to make explicit before,
 149 not after, benchmarking.

150 The formulation makes two falsifiable predictions. First, reducing residency while
 151 holding the access pattern fixed should expose a higher memory-access cost. Second,
 152 when $R_C \geq \theta_C$, configurations with $R_B < 1$ must expose transfer time even under
 153 ideal compute-transfer overlap. It does not, without additional measurements, predict
 154 the magnitude of a strategy’s speed-up or claim that strategy rankings necessarily
 155 invert.

156 The two coordinates are jointly necessary but not sufficient for latency prediction.
 157 Equal (R_C, R_B) does not imply equal latency because absolute service times, access
 158 locality, contention, and synchronisation can differ. This non-identifiability is useful: it
 159 defines the residual phenomena that must be measured rather than silently attributed
 160 to capacity or bandwidth.

Table 1 Selected measured points from the compiled CPU pointer-chain experiment. Values are mean $\pm$ one standard deviation over seven trials.

Working set (MiB)	R_C	Latency (ns/load)	Interpretation
0.5	96.000	6.8 ± 0.1	small working set
48	1.000	118.5 ± 6.6	reported LLC capacity
96	0.500	132.5 ± 14.3	residency convention
256	0.188	169.7 ± 10.4	largest working set

2.3 CPU proof-of-concept measurements

We evaluated working-set sensitivity on an Intel Xeon Platinum 8370C CPU exposed through a virtualised Linux environment. The experiment used a randomised pointer chain over working sets from 0.5 to 256 MiB and seven trials per point. A compiled C loop performed dependent loads and timed them with `CLOCK_MONOTONIC_RAW`; Python was used only to construct the chain and summarise trials. Linux sysfs reported a 48 MiB last-level cache, which we used as C_{fast} . Mean dependent-load time was 6.8 ± 0.1 ns at 0.5 MiB ($R_C = 96$), 118.5 ± 6.6 ns at 48 MiB ($R_C = 1$), 132.5 ± 14.3 ns at 96 MiB ($R_C = 0.5$), and 169.7 ± 10.4 ns at 256 MiB ($R_C = 0.188$), where uncertainty is one standard deviation across trials. The non-monotonic intermediate points show that cache level, translation, virtualisation, and system noise matter. In particular, these data do not show a discontinuity at the conventional $R_C = 0.5$ label.

As a separate harness check, we used the layered matrix program with 12 layers, $d = 512$, batch size 8, and five timing windows. It varied the fraction of layer weights retained in the fast tier. The mean total latency averaged over the three sampled points below $R_C = 0.5$ was 4.03 ms, compared with 1.87 ms for the fully resident point, a 2.15-fold descriptive ratio. Because computation and copying share the same CPU memory system, the accompanying R_B sweep was noisy and is not used as evidence for the I/O boundary.

These measurements are proof-of-concept results from one CPU, not a substitute for the previously planned A100, MI250, TPU v4, Inferentia2, and Optane campaign. Accordingly, we make no quantitative cross-accelerator, classifier- accuracy, energy, or strategy-inversion claim in this version.

2.4 Accelerator proof-of-concept measurements

We ran the layered program on 3 real accelerators of distinct architectures and two vendors—an NVIDIA Tesla T4, an NVIDIA A100, and a Google TPU v5e—with $d = 2048$, 24 layers, batch size 8, and ten timing windows. On the CUDA devices, computation and host-to-device transfer were timed separately with CUDA events; non-resident layer weights were staged in pinned host memory and copied per step, so residency (R_C) and overlap (R_B) were varied by construction. These runs test the two falsifiable predictions of Section 2.2 on hardware whose device–host link, unlike a single CPU’s memory system, makes the exposed-transfer regime physically reachable.

Prediction 1 (residency) holds on all 3 accelerators. On every device, mean per-step latency in the capacity-limited region ($R_C < \theta_C$) exceeded that at full residency

Table 2 Capacity crossover across 3 accelerators (layered probe, $d = 2048$). T is mean per-step latency; “Ratio” is capacity-limited over resident. The direction (ratio > 1) is the predicted effect; magnitudes are device-dependent and not predicted.

Accelerator	Resident T (ms)	Cap.-limited T (ms)	Ratio	Overlap sweep
Tesla T4	14.5	41.1	$2.83\times$	overlap boundary resolved
NVIDIA A100	52.4	70.6	$1.35\times$	sweep B compute-bound
Google TPU v5e	220.6	270.6	$1.23\times$	R_B split unreliable (XLA)

($R_C \geq 1$), by 1.23 – $2.83\times$ (Table 2). The formulation predicts this *direction*, not its magnitude; the magnitude shrinks on higher-bandwidth devices, where the offload penalty is smaller relative to compute.

Latency is monotone in the number of offloaded (transferred) layers down to two offloaded layers on all three devices; the only departure is the single fully-resident point ($R_C = 1$, zero transfers), which sits above the trend on the A100 (+9 ms) and the TPU (+42 ms) but not on the slower T4. We traced this to a measurement artifact rather than device memory behaviour: the fully-resident point is the one operating point at which the offload code path is skipped entirely, so it executes a structurally distinct computation—on the TPU a separately compiled XLA graph. At $d = 2048$ the true per-step memory signal is small on the fast accelerators, so this point’s one-off compilation and launch overhead exceed the residual signal, whereas on the slower T4 the signal dominates and monotonicity is preserved. An eager-execution control (NumPy backend, identical sweep) shows the fully-resident point as the *lowest* latency, confirming the effect originates in the accelerator execution/timing path, not the probe logic. We have since added a discarded warm-up iteration to the harness so that per-point compilation is excluded from future timings.

Prediction 2 (overlap) is resolved on the T4. Holding residency fixed and scaling compute moved the T4 operating point across the derived boundary $\theta_B = 1$: every point with $R_B < 1$ was labelled I/O-limited and every point with $R_B \geq 1$ coordination-dominated, with the transition between $R_B = 0.90$ and $R_B = 1.19$ —not at a fitted value. On the A100 the controlled sweep stayed compute-bound ($R_B > 2$) and so did not probe the boundary, and on the TPU the XLA path’s deferred execution makes the per-op R_B split unreliable; both need a larger-workload, preregistered run. The overlap boundary is therefore established on one GPU, while the capacity direction is established across three accelerators.

These are proof-of-concept results; they confirm the residency prediction across architectures and the overlap boundary on one GPU, but do not establish per-device boundary values, strategy inversion, classifier accuracy, or energy claims, which still require a larger, preregistered multi-platform campaign.

2.5 Reproducibility boundary

The repository contains the CPU raw JSONL records, per-accelerator JSON summaries, machine-readable aggregate summaries, and export scripts used for Tables 1

228 and 2. It also contains a simulated backend for testing the analysis pipeline. Simu-
 229 lated output is labelled as such and is not used as empirical evidence. The accelerator
 230 summaries support only the proof-of-concept comparisons reported here; they are not
 231 a substitute for raw event traces or independent hardware replication.

232 3 Discussion

233 The two-ratio description separates two questions that are often conflated in
 234 hierarchical-memory optimisation: whether the active data can reside in the fast tier,
 235 and whether compulsory transfer can be hidden behind computation. The resulting
 236 labels are operational rather than universal phases. $\theta_B = 1$ follows exactly from the
 237 overlap definition, whereas $\theta_C = 0.5$ is a declared midpoint that makes “majority resi-
 238 dent” precise. A different application may choose another residency threshold without
 239 changing Equation (3).

240 The CPU probes show that latency depends strongly on working-set residency, but
 241 they do not validate a sharp boundary at $R_C = 0.5$. They stress different mechanisms:
 242 dependent pointer loads probe cache and translation effects, while the layered matrix
 243 harness combines weight copying with computation. The accelerator runs (Section 2.4)
 244 exercise real DMA and asynchronous CUDA streams across three devices of two ven-
 245 dors. The residency prediction holds on all three; the overlap crossover at $\theta_B = 1$ is
 246 resolved on the T4, while the A100 sweep stayed compute-bound and the TPU’s XLA
 247 timing is unreliable. The data do not establish per-device boundary *values*, HBM- or
 248 collective-communication effects at scale, or the generality of the three labels across
 249 model families.

250 The framework is most useful as a measurement discipline. Reporting R_C and R_B
 251 alongside latency makes hidden assumptions about residency and overlap inspectable.
 252 It also prevents a common circularity: using total step duration in both the numerator
 253 of a bandwidth ratio and as the outcome being explained. For deployment, C_{fast} , W ,
 254 D , sustained bandwidth, and isolated compute time must be measured for the actual
 255 workload and hardware rather than copied from nominal data-sheet values.

256 Its broader relevance is methodological rather than tied to one model family.
 257 Weights, KV caches, embeddings, feature maps, and retrieval indices are different
 258 objects, but each admits the same audit: declare the active bytes and usable fast tier,
 259 measure compulsory cross-tier bytes and sustained bandwidth, and isolate compute
 260 time. Equations (4) and (5) then state which part of the pressure cannot be removed
 261 by placement or ideal overlap. This creates a common reporting layer under other-
 262 wise incomparable offloading, compression, scheduling, and memory- pooling studies.
 263 Whether it predicts application outcomes is a hypothesis for production workloads,
 264 not a result of the present probes.

265 The evidence changes one practical conclusion: capacity and transfer pressure
 266 should not be inferred from latency alone. A defensible comparison should report the
 267 working-set definition, usable fast capacity, compulsory bytes per step, sustained link
 268 bandwidth, isolated compute and transfer times, continuous (R_C, R_B) , and end-to-
 269 end latency. Those fields make denominator choices and uncontrolled changes visible,

allow threshold sensitivity to be recomputed, and separate a failed directional prediction from a merely small effect. This reporting contract is the principal reusable output of the study.

Several limitations are material. The present evidence comes from one CPU and three accelerators (two GPUs and a TPU) with small synthetic probes; it does not include production models, concurrent requests, accelerator energy measurements, strategy comparisons, or unseen-hardware classifier tests. The standard deviations describe repeated trials on one machine and are not confidence intervals over a population of platforms. These measurements are exploratory technical checks rather than independent hardware replicates, and no causal inference is supported. The pointer-chain endpoints also span several hierarchy transitions, so their ratio should not be interpreted as a universal effect size. Finally, Equation (2) is an accounting identity only when its terms are defined as non-overlapping critical-path contributions; the max-form lower bound avoids assuming that decomposition in measured systems.

A decisive follow-up should preregister operating-point grids around $R_C = 0.5$ and $R_B = 1$, collect raw event traces on at least two accelerator architectures, and compare fixed orchestration policies without changing other workload parameters. It should report continuous coordinates and repeat the analysis across plausible residency conventions rather than optimise a threshold on the evaluation data. Such evidence could test boundary sharpness, policy ranking, and generalisation. Until then, the contribution is the identifiable coordinate, its bounds and reporting contract, and a CPU-and-multi-accelerator falsification attempt—not universal accelerator validation.

4 Methods

Definitions and classification

We calculate $R_C = C_{\text{fast}}/W$ from a declared usable fast-tier capacity and active working-set size. We calculate $R_B = T_{\text{comp}}/T_{\text{transfer}}$, with compute and transfer timed separately. Classification gives capacity precedence: $R_C < 0.5$ is capacity-limited; otherwise $R_B < 1$ is I/O-limited; all remaining points are coordination-dominated. This rule is implemented in the released Python code. We retain the continuous ratios in every record because 0.5 is a reporting convention rather than an estimated device boundary. Any reuse should declare W and usable C_{fast} , report the unthresholded ratios, and repeat categorical summaries under application-relevant residency cut-offs when the conclusion depends on the label.

Pointer-chain capacity probe

The CPU probe allocates a NumPy array and constructs a seeded random permutation. A C11 kernel compiled with `-O3` traverses it as a dependent pointer chain; each loaded index determines the address of the next load, limiting vectorisation, instruction-level parallelism, and prefetching. We measured working sets of 0.5, 1, 2, 4, 8, 16, 24, 32, 48, 64, 96, 128, 192, and 256 MiB. Each point used seven timed trials after warm-up. The output stores working-set size, R_C , mean nanoseconds per dependent

311 load, standard deviation, and trial count. The fast-tier capacity was read from Linux
312 sysfs and was 48 MiB. The host exposed three logical CPUs of an Intel Xeon Platinum
313 8370C under KVM; the summary records the kernel, Python, and NumPy versions.
314 Timing used `clock_gettime(CLOCK_MONOTONIC_RAW)` inside the compiled loop. No
315 outlier was removed and CPU affinity or frequency was not controlled.

316 Layered matrix and copy probe

317 The second probe allocates square, float32 layer matrices; a selected fraction remains
318 resident and non-resident matrices are copied before multiplication. Matrix entries
319 are deterministically seeded and scaled by $1/\sqrt{d}$ to prevent numerical overflow during
320 repeated multiplication. On the CPU we used NumPy’s wall-clock path with 12 layers,
321 $d = 512$, and five windows per point. On the accelerators (Section 2.4) the same
322 program ran with 24 layers, $d = 2048$, batch size 8, ten windows, and pinned host
323 staging: on the NVIDIA Tesla T4 and A100 through the harness’s CUDA-event path,
324 timing computation and host-to-device transfer separately with `torch.cuda.Event`;
325 and on a Google TPU v5e through the `torch.xla` path. Because XLA executes lazily,
326 per-op R_B timing on the TPU is unreliable and we treat its R_B split as indicative
327 only, relying on end-to-end latency for the residency test.

328 For the reported capacity ratio, we averaged total latency at the three sampled
329 points below $R_C = 0.5$ and divided by latency at $R_C = 1$. This endpoint summary is
330 descriptive; no hypothesis test or population-level confidence interval is claimed. The
331 pointer-chain table reports mean and population standard deviation across the seven
332 repeated trials exactly as stored in the JSON summary.

333 Software and provenance

334 The compiled pointer-chain run used Python 3.12.13, NumPy 2.5.2, and GCC 13.3;
335 the earlier layered-matrix harness check used Python 3.11.15, but that summary did
336 not capture the NumPy version. Seeds, grids, timing code, raw records, summaries,
337 and commands are version controlled. The analytical regime map reads threshold
338 constants from the same module as the classifier. The simulated backend is provided
339 solely for software testing and qualitative exploration; its outputs are excluded from
340 the reported measurements.

341 Statistics and reporting

342 This proof-of-concept study did not use random assignment, blinding, or a sample-size
343 calculation. Trial counts were fixed before summarisation. We report all measured grid
344 points in the machine-readable data, selected points in Table 1, arithmetic means, and
345 one standard deviation. No claim of statistical significance is made. Hardware-platform
346 generalisation requires independent machines and is left for future work.

347 Data availability

348 All data supporting the reported CPU and accelerator measurements are
349 included in the public repository at <https://github.com/leemgs/orion>, under

350 `code/results/cpu_probe/` and `code/results/colab_probe/`. These directories con-
351 tain the raw JSONL records and machine-readable JSON summaries used in the text
352 and tables.

353 Code availability

354 The analysis and measurement code is available at <https://github.com/leemgs/orion>.
355 The `code/experiments/` directory contains the CPU probes, analytical-figure gener-
356 ator, and optional CUDA/XLA measurement paths. Exact reproduction commands
357 are provided in the README and Supplementary Information. A separately labelled
358 simulated backend is supplied for software testing; simulated output is not used as
359 evidence in this article.

360 Author Contributions

361 G.L.: Conceptualization, Formal analysis, Investigation, Methodology, Software, Val-
362 idation, Visualization, Writing – original draft, Writing – review & editing.

363 Competing Interests

364 The author declares no competing interests.

365 Acknowledgements

366 We thank Donghyeon Kang, Seungmin Oh and Yunsu Lee for valuable feedback. We
367 are also grateful to Wooyeon Lee, Hyoyoung Kim, and Heekuk Lee, experts in systems
368 and memory, whose insights and meaningful comments greatly improved this work.

369 References

- 370 [1] Xiaoyu Ma and David Patterson. Challenges and research directions for large
371 language model inference hardware. *IEEE Computer*, 59, 2026. URL <https://arxiv.org/abs/2601.05047>. Accepted.
372
373 [2] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney,
374 and Kurt Keutzer. Ai and memory wall. *IEEE Micro*, 2024. URL <https://arxiv.org/abs/2403.14123>.
375
376 [3] International Energy Agency. Electricity 2024: Analysis and forecast to 2026.
377 Technical report, IEA, 2024. URL <https://www.iea.org/reports/electricity-2024>.
378 Global average carbon intensity: 494 gCO₂/kWh.
379 [4] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse.
380 DynamoLLM: Designing LLM inference clusters for performance and energy
381 efficiency. In *Proc. HPCA*, 2025. URL <https://arxiv.org/abs/2408.00741>.
382 [5] Isabel Juniewicz. Hyperscaler capex is on trend to outpace their cash inflows by
383 the end of 2026. Epoch AI, Data Insight, 2026. Accessed: 2026-06-21. [Online].
384 Available: <https://epoch.ai/data-insights/hyperscaler-capex-vs-cash-flow>.

- [6] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL <https://arxiv.org/abs/2407.21783>.
- [7] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024. URL <https://arxiv.org/abs/2401.04088>.
- [8] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *Proc. ICLR*, 2021. URL <https://arxiv.org/abs/2010.11929>.
- [9] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *Proc. ICML*, 2023. URL <https://arxiv.org/abs/2301.12597>.
- [10] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019. URL <https://arxiv.org/abs/1702.08734>.
- [11] Zixuan Zhou, Xuefei Ning, Ke Hong, Tianyu Fu, Jiaming Xu, Shiyao Li, Yuming Lou, Luning Wang, Zhihang Yuan, Xiuhong Li, Shengen Yan, Guohao Dai, Xiaoping Zhang, Yuhang Dong, and Yu Wang. A survey on efficient inference for large language models. *arXiv preprint arXiv:2404.14294*, 2024. URL <https://arxiv.org/abs/2404.14294>.
- [12] Li Baolin et al. LLM inference serving: Survey of recent advances and opportunities. In *Proc. HPEC*, 2024. URL <https://arxiv.org/abs/2407.12391>.
- [13] Miao Xupeng et al. Towards efficient generative large language model serving: A survey from algorithms to systems. *ACM Computing Surveys*, 58(1):1–37, 2025. URL <https://arxiv.org/abs/2312.15234>.
- [14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. *arXiv preprint arXiv:2309.06180*, 2023. URL <https://arxiv.org/abs/2309.06180>.
- [15] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *Proc. OSDI*, pages 521–538, 2022. URL <https://www.usenix.org/conference/osdi22/presentation/you>.
- [16] Du Kuntai et al. Prefillonly: An inference engine for prefill-only workloads in large language model applications. In *Proc. SOSP*, 2025. URL <https://arxiv.org/abs/2505.07203>.
- [17] Jin Yibo et al. P/d-serve: Serving disaggregated large language model at scale. *arXiv preprint arXiv:2408.08147*, 2024. URL <https://arxiv.org/abs/2408.08147>.
- [18] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. Flexgen: High-throughput generative inference of large language models with a single gpu. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. URL <https://arxiv.org/abs/2303.06865>.

- [19] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Chenyang Li, Dong Li, Erich Zheng, Olatunji Ruwase, and Yuxiong He. Deep-speed inference: Enabling efficient inference of transformer models at unprecedented scale. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2022. URL <https://arxiv.org/abs/2207.00032>.
- [20] Xu Yi et al. Pie: Pooling CPU memory for LLM inference. *arXiv preprint arXiv:2411.09317*, 2024. URL <https://arxiv.org/abs/2411.09317>.
- [21] Xuanlin Jiang et al. Neo: Saving GPU memory crisis with CPU offloading for online llm inference. *arXiv preprint arXiv:2411.01142*, 2024. URL <https://arxiv.org/abs/2411.01142>.
- [22] Xiangwen Zhuge et al. SpecOffload: Unlocking latent GPU capacity for LLM inference on resource-constrained devices. *arXiv preprint arXiv:2505.10259*, 2025. URL <https://arxiv.org/abs/2505.10259>.
- [23] Xu Jiale et al. eLLM: Elastic memory management framework for efficient LLM serving. *arXiv preprint arXiv:2506.15155*, 2025. URL <https://arxiv.org/abs/2506.15155>.
- [24] Chien-Chin Huang, Gu Jin, and Jinyang Li. Swapadvisor: Pushing deep learning beyond the gpu memory limit via smart swapping. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020. URL <https://doi.org/10.1145/3373376.3378530>.
- [25] Wang Kun et al. Swapnet: Efficient swapping for dnn inference on edge ai devices beyond the memory budget. *IEEE Transactions on Mobile Computing*, 23(9): 8935–8950, 2024. URL <https://arxiv.org/abs/2401.16757>.
- [26] Kida Shun et al. Revisiting memory swapping for big-memory applications. In *Proc. HPCASIA*, 2025. URL <https://doi.org/10.1145/3712031.3712332>.
- [27] Elias Frantar and Dan Alistarh. SparseGPT: Massive language models can be accurately pruned in one-shot. In *Proc. ICML*, 2023. URL <https://arxiv.org/abs/2301.00774>.
- [28] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Proc. NeurIPS*, 2022. URL <https://arxiv.org/abs/2205.14135>.
- [29] Korthikanti Vijay Anand et al. Reducing activation recomputation in large transformer models. *Proc. MLSys*, 5:341–353, 2023. URL <https://arxiv.org/abs/2205.05198>.
- [30] Huang Yanpin et al. GPipe: Efficient training of giant neural networks using pipeline parallelism. In *Proc. NeurIPS*, 2019. URL <https://arxiv.org/abs/1811.06965>.
- [31] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019. URL <https://arxiv.org/abs/1909.08053>.
- [32] A.H. Hunter et al. Beyond malloc efficiency to fleet efficiency: a hugepage-aware memory allocator. In *Proc. OSDI*, 2021. URL <https://www.usenix.org/>

- conference/osdi21/presentation/hunter.
- [33] Yang Hanmei et al. Numalloc: A faster numa memory allocator. In *Proc. ISMM*, 2023. URL <https://doi.org/10.1145/3591195.3595276>.
- [34] Ugljesa Milic, Oreste Villa, Evgeny Bolotin, Akhil Arunkumar, Eiman Ebrahimi, Aamer Jaleel, Alex Ramirez, and David Nellans. Beyond the socket: NUMA-aware GPUs. In *Proc. MICRO*, pages 123–135, 2017. URL <https://doi.org/10.1145/3123939.3124534>.
- [35] Carl A. Waldspurger. Memory resource management in VMware ESX Server. In *Proceedings of the 5th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 181–194, 2002. URL <https://www.usenix.org/conference/osdi-02/memory-resource-management-vmware-esx-server>.
- [36] Long Sifan et al. A survey on intelligent network operations and performance optimization based on large language models. *IEEE Communications Surveys & Tutorials*, 2025. URL <https://ieeexplore.ieee.org/document/10829820>.
- [37] Zhi Wang et al. Model offloading techniques for edge AI: A survey. *IEEE Internet of Things Journal*, 2023. URL <https://doi.org/10.1109/JIOT.2023.3254417>. DOI: 10.1109/JIOT.2023.3254417.
- [38] Sarkar Rishov et al. Edge-moe: Memory-efficient multi-task vision transformer architecture with task-level sparsity via mixture-of-experts. In *Proc. ICCAD*, 2023. URL <https://arxiv.org/abs/2305.18691>.
- [39] Luo Haoxiang et al. Toward edge general intelligence with multiple-large language model (Multi-LLM): architecture, trust, and orchestration. *IEEE Trans. Cogn. Commun. Netw.*, 2025. URL <https://arxiv.org/abs/2507.00672>.
- [40] He Zixuan et al. CPU–GPU heterogeneous computation offloading and resource allocation scheme for industrial internet of things. *IEEE Internet of Things Journal*, 11(6):11152–11164, 2023. URL <https://ieeexplore.ieee.org/document/10319405>.
- [41] Beaumont Olivier et al. Optimal GPU-CPU offloading strategies for deep neural network training. In *Proc. Euro-Par*, 2020. URL https://doi.org/10.1007/978-3-030-57675-2_10.
- [42] R. L. Mattson, J. Gecsei, D. R. Slutz, and I. L. Traiger. Evaluation techniques for storage hierarchies. *IBM Systems Journal*, 9(2):78–117, 1970. doi: 10.1147/sj.92.0078. URL <https://doi.org/10.1147/sj.92.0078>.
- [43] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785. URL <https://doi.org/10.1145/1498765.1498785>.
- [44] Leonard Kleinrock. *Queueing Systems, Volume 1: Theory*. John Wiley & Sons, New York, 1975. URL <https://dl.acm.org/doi/book/10.5555/1096491>. ISBN 978-0-471-49110-1.